

CO1-AT3-Coding Challenge

View Only

← Back

QUESTIONS

Q1



Smart University Payroll Management System

25 marks

Q2



Smart Vehicle Insurance Claim System

25 marks

2/2 answered

Q1 Smart University Payroll Management System

25 marks

A university employs three categories of staff: Teaching Faculty, Technical Staff, and Visiting Faculty. The payroll system should calculate the monthly salary based on the employee type while maintaining a common employee hierarchy. Requirements Create a superclass Employee with the following private data members: employeeId employeeName department basicSalary Provide: Default constructor Parameterized constructor Getter and Setter methods displayEmployee() calculateSalary() Create the following subclasses: TeachingFaculty teachingHours incentivePerHour Salary: basicSalary + (teachingHours × incentivePerHour) TechnicalStaff overtimeHours overtimeRate Salary: basicSalary + (overtimeHours × overtimeRate) VisitingFaculty lecturesHandled paymentPerLecture Salary: lecturesHandled × paymentPerLecture Override calculateSalary() in all subclasses. Create an array of Employee objects and demonstrate runtime polymorphism. Display: Employee details Monthly salary Highest-paid employee Average salary Search an employee using the employee ID.

Test Case 1 (Normal Case)

Input

3

TeachingFaculty

101

Dr. Arun

CSE

50000

20

500

TechnicalStaff

102

Mr. Ravi

ECE

35000

15

300

VisitingFaculty

103

Dr. Kumar

IT

0

40

1200

Search Employee ID:

102

Expected Output

Employee Details

Employee ID : 101

Name : Dr. Arun

Department : CSE

Monthly Salary : 60000.0

Employee ID : 102
Name : Mr. Ravi
Department : ECE
Monthly Salary : 39500.0

Employee ID : 103
Name : Dr. Kumar
Department : IT
Monthly Salary : 48000.0

Highest Paid Employee

45100.07

Employee Found
Employee ID : 102
Name : Mr. Ravi
Salary : 39500.0

Your Code

```
1 import java.util.Scanner;  
2  
3 class Employee {  
4     private int employeeId;  
5     private String employeeName;  
6     private String department;  
7     private double basicSalary;
```

Input

3
TeachingFaculty
101
Dr. Arun

```

8     public Employee() {}
9     public Employee(int employeeId, String employeeName, String department, double basicSalary) {
10         this.employeeId = employeeId;
11         this.employeeName = employeeName;
12         this.department = department;
13         this.basicSalary = basicSalary;
14     }
15     public int getEmployeeId() { return employeeId; }
16     public String getEmployeeName() { return employeeName; }
17     public String getDepartment() { return department; }
18     public double getBasicSalary() { return basicSalary; }
19     public void setEmployeeId(int employeeId) { this.employeeId = employeeId; }
20     public void setEmployeeName(String employeeName) { this.employeeName = employeeName; }
21     public void setDepartment(String department) { this.department = department; }
22     public void setBasicSalary(double basicSalary) { this.basicSalary = basicSalary; }
23     public double calculateSalary() {
24         return basicSalary;
25     }
26     public void displayEmployee() {
27         System.out.println("Employee ID : " + employeeId);
28         System.out.println("Name : " + employeeName);
29         System.out.println("Department : " + department);
30         System.out.println("Monthly Salary : " + calculateSalary());
31     }
32     public static void main(String[] args) {
33         Scanner sc = new Scanner(System.in);
34         int n = Integer.parseInt(readNonEmptyLine(sc));
35         Employee[] employees = new Employee[n];
36         for (int i = 0; i < n; i++) {
37             String type = readNonEmptyLine(sc);
38             int id = Integer.parseInt(readNonEmptyLine(sc));
39             String name = readNonEmptyLine(sc);
40             String dept = readNonEmptyLine(sc);

```

Output

Employee Details

Employee ID : 101

Name : Dr. Arun

Department : CSE

Monthly Salary :

```
41         double basicSalary = Double.parseDouble(
42             type.equals("TeachingFaculty")) {
43             double teachingHours = Double.parseDouble(
44                 type.equals("TeachingFaculty")) {
45                 employees[i] = new TeachingFaculty(
46                     type.equals("TechnicalStaff") {
47                         double overtimeHours = Double.parseDouble(
48                             type.equals("TechnicalStaff") {
49                             employees[i] = new TechnicalStaff(
50                                 type.equals("VisitingFaculty") {
51                                     double lecturesHandled = Double.parseDouble(
52                                         type.equals("VisitingFaculty") {
53                                             employees[i] = new VisitingFaculty(
54                                                 type.equals("VisitingFaculty") {
55                                                     }
56                                                     System.out.println("Employee Details");
57                                                     for (Employee e : employees) {
58                                                         System.out.println();
59                                                         e.displayEmployee();
60                                                     }
61                                                     System.out.println();
62                                                     Employee highestPaid = employees[0];
63                                                     for (Employee e : employees) {
64                                                         if (e.calculateSalary() > highestPaid.calculateSalary())
65                                                             highestPaid = e;
66                                                     }
67                                                     }
68                                                     System.out.println("Highest Paid Employee");
69                                                     System.out.println("Employee ID : " + highestPaid.getId());
70                                                     System.out.println("Salary : " + highestPaid.calculateSalary());
71                                                     System.out.println();
72                                                     double total = 0;
73                                                     for (Employee e : employees) total += e.calculateSalary();
```

```
74     double average = total / n;
75     System.out.println("Average Salary");
76     System.out.printf("%.2f%n", average);
77     System.out.println();
78     String line = readNonEmptyLine(sc);
79     int searchId;
80     if (line.contains("Search Employee ID")) {
81         searchId = Integer.parseInt(readNonEmptyLine(sc));
82     } else {
83         searchId = Integer.parseInt(line);
84     }
85     boolean found = false;
86     for (Employee e : employees) {
87         if (e.getEmployeeId() == searchId) {
88             System.out.println("Employee Found");
89             System.out.println("Employee ID : " + e.getEmployeeId());
90             System.out.println("Name : " + e.getName());
91             System.out.println("Salary : " + e.getSalary());
92             found = true;
93             break;
94         }
95     }
96     if (!found) {
97         System.out.println("Employee Not Found");
98     }
99
100     sc.close();
101 }
102 private static String readNonEmptyLine(Scanner sc) {
103     while (sc.hasNextLine()) {
104         String s = sc.nextLine().trim();
105         if (!s.isEmpty()) return s;
106     }
```

```

107         return "";
108     }
109 }
110
111 class TeachingFaculty extends Employee {
112     private double teachingHours;
113     private double incentivePerHour;
114     public TeachingFaculty(int employeeId, String employeeName,
115                             double basicSalary, double incentivePerHour) {
116         super(employeeId, employeeName, department, basicSalary);
117         this.teachingHours = teachingHours;
118         this.incentivePerHour = incentivePerHour;
119     }
120
121     @Override
122     public double calculateSalary() {
123         return getBasicSalary() + (teachingHours * incentivePerHour);
124     }
125 }
126
127 class TechnicalStaff extends Employee {
128     private double overtimeHours;
129     private double overtimeRate;
130     public TechnicalStaff(int employeeId, String employeeName,
131                             double basicSalary, double overtimeHours,
132                             double overtimeRate) {
133         super(employeeId, employeeName, department, basicSalary);
134         this.overtimeHours = overtimeHours;
135         this.overtimeRate = overtimeRate;
136     }
137     @Override
138     public double calculateSalary() {
139         return getBasicSalary() + (overtimeHours * overtimeRate);
140     }
141 }

```

```
140 }
141 class VisitingFaculty extends Employee {
142     private double lecturesHandled;
143     private double paymentPerLecture;
144     public VisitingFaculty(int employeeId, String employeeName,
145                             double basicSalary, double paymentPerLecture) {
146         super(employeeId, employeeName, department,
147               this.lecturesHandled = lecturesHandled;
148               this.paymentPerLecture = paymentPerLecture;
149     }
150     @Override
151     public double calculateSalary() {
152         return getBasicSalary() + (lecturesHandled * paymentPerLecture);
153     }
154 }
```



✓ Submitted